Spring Boot Filters

Advanced Concepts and Best Practices

Study notes: response headers, response commitment, direct error responses, wrappers, request modification, and OncePerRequestFilter.

> **Diagram rule: All diagrams in this document flow strictly from top to bottom. No boxes or left-to-right flow diagrams are used.**

1. Revision: Request–Response Flow

A filter operates before the request reaches the DispatcherServlet. Multiple filters can be chained together.

Request flow

```
Client
   |
   v
Filter 1
   |
   v
Filter 2
   |
   v
Filter 3
   |
   v
DispatcherServlet
   |
   v
Controller
   |
   v
Service
   |
   v
Repository
```

Response flow

```
Repository
   |
   v
Service
   |
   v
Controller
   |
   v
DispatcherServlet
   |
   v
Filter 3
   |
   v
Filter 2
   |
   v
Filter 1
   |
   v
Client
```

- DispatcherServlet is Spring MVC's central servlet.
- Filters belong to the Servlet API (jakarta.servlet), not specifically to the Spring Framework.
- Spring can manage filter instances using @Component.
- An Interceptor operates at the Spring MVC level, between DispatcherServlet and handler execution.

2. Modifying the HTTP Response

An HTTP response has three main parts:

- Status code: e.g., 200 OK or 401 Unauthorized.
- Headers: metadata such as content type and request ID.
- Body: response data, such as JSON.

Adding a response header

Example header: `X-Request-ID: a-random-generated-value`

```
@Component
public class ResponseHeaderFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request,
                         ServletResponse response,
                         FilterChain chain)
            throws IOException, ServletException {

        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        String requestId = UUID.randomUUID().toString();
        httpResponse.setHeader("X-Request-ID", requestId);

        chain.doFilter(request, response);
    }
}
```

Required imports include the Servlet API types, HttpServletResponse, UUID, and IOException.

- UUID.randomUUID() generates a UUID; toString() converts it to text.
- setHeader() sets a header and replaces an existing value for that header name. addHeader() can add another value.

Key takeaway: Set ordinary response headers before calling chain.doFilter() when possible.

3. Why Set Headers Before chain.doFilter()?

```
Client
   |
   v
Response Header Filter
   |
   v
Set X-Request-ID
   |
   v
chain.doFilter()
   |
   v
DispatcherServlet
   |
   v
Controller
   |
   v
Response is generated
   |
   v
Client receives response with header
```

The filter sets the header before downstream processing generates and commits the response.

Response commitment

A response becomes committed when the container has progressed to sending the status, headers, and body to the client. After commitment, changing headers or status is generally too late.

```
Filter starts
   |
   v
Set response headers
   |
   v
Call chain.doFilter()
   |
   v
Controller generates response
   |
   v
DispatcherServlet writes response
   |
   v
Response may become committed
   |
   v
Filter execution resumes
   |
   v
Do not assume headers/body can still be changed
```

The response is not necessarily committed immediately when chain.doFilter() returns. The exact point depends on response handling. Do not rely on modifying an already-generated response.

4. Returning a Response Directly from a Filter

A filter can reject a request and return its own response without allowing it to reach the controller. This is useful for authentication checks and some request-rejection scenarios.

```
@Component
public class AuthenticationFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request,
                         ServletResponse response,
                         FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;
        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        String token = httpRequest.getHeader("Token");

        if (token == null || !token.equals("12345")) {
            httpResponse.setStatus(
                    HttpServletResponse.SC_UNAUTHORIZED);
            httpResponse.setContentType("application/json");
            httpResponse.getWriter().write(
                    "{\"message\":\"Authentication is required\"}");
            return;
        }

        chain.doFilter(request, response);
    }
}
```

The hardcoded token is only a demonstration of filter mechanics. Real authentication should use an appropriate authentication mechanism.

Invalid-token flow

```
Client sends request
   |
   v
Authentication Filter
   |
   v
Read and validate Token
   |
   v
Token invalid
   |
   v
Set status to 401
   |
   v
Write JSON error body
   |
   v
return
   |
   v
Client receives error response
```

Important: return prevents this filter from calling chain.doFilter(), so the request does not proceed through the remaining chain from this point.

5. Setting Status, Content Type, and Body

Status code

```
httpResponse.setStatus(
        HttpServletResponse.SC_UNAUTHORIZED);
```

Alternatively, sendError(HttpServletResponse.SC_UNAUTHORIZED) invokes the servlet container's error-response handling. It is not identical to simply calling setStatus().

Content type

```
httpResponse.setContentType("application/json");
```

Response body

```
httpResponse.getWriter().write(
        "{\"message\":\"Authentication is required\"}"
);
```

getWriter() provides a writer for character data. A filter can directly return a status, headers, and body when rejecting a request.

> **Remember: A direct error response is different from modifying a response that the controller has already generated.**

6. Why a Filter Cannot Easily Modify an Existing Response Body

Suppose a controller returns:

```
{
  "name": "Aditya",
  "message": "Student is saved successfully"
}
```

You want to add an appName field:

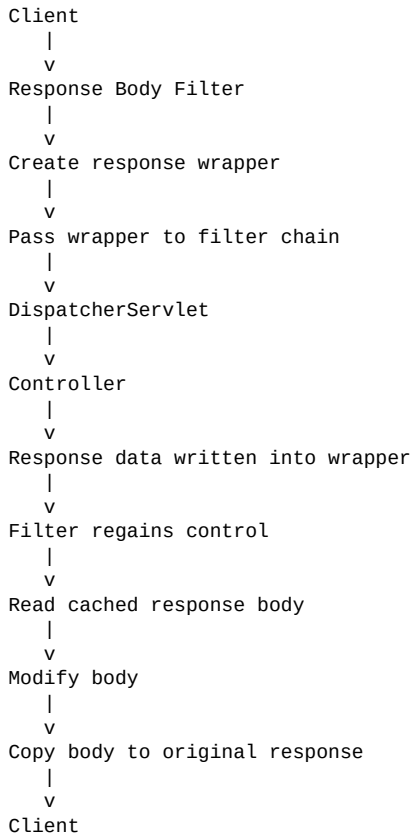
```
{
  "name": "Aditya",
  "message": "Student is saved successfully",
  "appName": "Student Management System"
}
```

- The controller returns response data.
- DispatcherServlet processes that data and writes the response.
- The response may already be committed when execution returns to the filter.
- The filter does not automatically receive the controller's response as an editable Java object.

Calling chain.doFilter() and then writing to the original response is not a reliable body-modification strategy.

7. The Wrapper Approach

Spring provides ContentCachingResponseWrapper in org.springframework.web.util. It can cache response content so the filter can inspect it before copying it to the underlying response.



The idea resembles the Decorator design pattern: a wrapper provides additional behavior while exposing a compatible interface.

- Downstream code writes through the wrapper.
- The filter can inspect the cached bytes after downstream processing.
- The filter can construct modified content and copy it to the actual response.

8. Implementing a Response Body Filter

```
@Component
public class ResponseBodyFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request,
                         ServletResponse response,
                         FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest httpRequest =
                (HttpServletRequest) request;
        HttpServletResponse httpResponse =
                (HttpServletResponse) response;

        ContentCachingResponseWrapper wrappedResponse =
                new ContentCachingResponseWrapper(httpResponse);

        chain.doFilter(httpRequest, wrappedResponse);

        byte[] originalBodyBytes =
                wrappedResponse.getContentAsByteArray();

        String originalBody = new String(
                originalBodyBytes, StandardCharsets.UTF_8);

        String modifiedBody = """
                {
                  "originalResponse": %s,
                  "appName": "Student Management System"
                }
                """.formatted(originalBody);

        wrappedResponse.resetBuffer();
        wrappedResponse.getWriter().write(modifiedBody);
        wrappedResponse.copyBodyToResponse();
    }
}
```

This example illustrates the wrapper technique. It wraps the original JSON inside originalResponse and adds appName; it does not create the flat JSON structure from the earlier example.

Implementation steps

- Create wrapper: wrap the original HttpServletResponse.
- Pass wrapper: call chain.doFilter(request, wrappedResponse).
- Read bytes: getContentAsByteArray() returns the cached content.
- Convert: use a defined charset such as UTF-8.
- Reset buffer: clear the cached body before writing the replacement.
- Write and copy: write modified content and call copyBodyToResponse().

Critical: Without copyBodyToResponse(), the cached response body may not reach the client.

9. Production Considerations and Best Practice

- The wrapper caches response content in memory; large responses can consume substantial memory.
- For JSON, parse and modify the object using a JSON library such as Jackson instead of concatenating strings.
- Account for content type and character encoding.
- Avoid this simple technique for binary content, streaming responses, and large downloads.
- If the body changes, ensure the final response length and relevant headers remain correct.
- Business response data should generally be created by the controller or service layer.

Why response-body rewriting is usually discouraged

- It can violate the API contract expected by clients.
- It creates hidden behavior that is not obvious from the controller.
- Disabling or changing the filter can unexpectedly change API behavior.
- Caching and copying bodies adds memory and processing overhead.

Recommendation: Use filters for cross-cutting concerns. Construct business response fields in a response DTO or service/controller layer.

10. Understanding HTTP Request Components

Component	Example	Purpose
HTTP method	POST	Describes the operation
Request URI	/api/students	Identifies the endpoint path
Headers	Token: 12345	Supplies metadata
Request parameters	?page=1	Supplies parameter values
Request body	JSON student data	Supplies payload data

```
String method = httpRequest.getMethod();
String uri = httpRequest.getRequestURI();
String token = httpRequest.getHeader("Token");
String page = httpRequest.getParameter("page");
```

These calls read details from the incoming request.

11. Can We Directly Modify an HTTP Request?

The standard HttpServletRequest API exposes getters for method, URI, headers, and parameters, but no general-purpose setters for replacing these incoming values.

```
httpRequest.getMethod();
httpRequest.getRequestURI();
httpRequest.getHeader("Token");
httpRequest.getParameter("page");
```

These are not standard HttpServletRequest methods:

```
httpRequest.setMethod("GET");
httpRequest.setRequestURI("/another-endpoint");
httpRequest.setHeader("Token", "new-value");
httpRequest.setParameter("page", "2");
```

The original request represents what the client sent. If downstream code needs modified request behavior, use an appropriate request wrapper.

12. Request Wrappers

HttpServletRequestWrapper lets a custom request delegate to the original request while overriding selected behavior.

```
Client sends request
   |
   v
Original HttpServletRequest
   |
   v
Create custom request wrapper
   |
   v
Override selected request methods
   |
   v
Pass wrapped request to filter chain
   |
   v
DispatcherServlet
   |
   v
Controller receives wrapped request
```

For example, a custom wrapper could override getHeader() to return a replacement value for a particular header.

A wrapper changes what downstream code sees when it calls overridden methods; it does not change what the client originally sent.

13. Request Body Modification and Caching

The request body is generally read through an input stream or reader. Once consumed, it cannot automatically be read again from the original stream.

- ContentCachingRequestWrapper caches content as downstream code reads it.
- It does not automatically make the request body freely replaceable before downstream processing.
- To replace the body, a custom HttpServletRequestWrapper may override getInputStream() and getReader().

```
Client sends request body
   |
   v
Filter receives original request
   |
   v
Create appropriate request wrapper
   |
   v
Wrapper exposes required body
   |
   v
Pass wrapped request to filter chain
   |
   v
DispatcherServlet reads request
   |
   v
Controller processes the body
```

Key distinction: Caching content is not the same as replacing the request body.

14. Spring's OncePerRequestFilter

OncePerRequestFilter is a Spring Web base class for filters that need its once-per-request handling behavior.

```
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class RequestLoggingFilter
        extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain)
            throws ServletException, IOException {

        System.out.println(
                "Method: " + request.getMethod());

        System.out.println(
                "URI: " + request.getRequestURI());

        filterChain.doFilter(request, response);
    }
}
```

- The request and response are already HTTP-specific types.
- No manual casts are needed inside doFilterInternal().
- It provides infrastructure for avoiding repeated execution within its defined dispatch semantics.

“Once per request” is not an unconditional guarantee across every possible dispatch or asynchronous scenario; behavior depends on dispatch type and configuration.

```
Incoming request
   |
   v
OncePerRequestFilter
   |
   v
doFilterInternal()
   |
   v
Execute filter logic
   |
   v
filterChain.doFilter()
   |
   v
Next filter or DispatcherServlet
```

15. The shouldNotFilter() Method

OncePerRequestFilter provides shouldNotFilter(HttpServletRequest request) so a subclass can skip selected requests.

```
@Override
protected boolean shouldNotFilter(
        HttpServletRequest request) {

    String uri = request.getRequestURI();

    return !uri.startsWith("/api/");
}
```

- Returns true: skip this filter.
- Returns false: execute the filter normally.
- In this example, /api/students is filtered, while /health and /actuator/info are skipped.

Skip flow

```
Incoming request
   |
   v
shouldNotFilter(request)
   |
   v
Should this request be skipped?
   |
   v
Yes
   |
   v
Skip this filter's internal logic
```

Normal flow

```
Incoming request
   |
   v
shouldNotFilter(request)
   |
   v
Return false
   |
   v
doFilterInternal()
   |
   v
Execute filter logic
   |
   v
filterChain.doFilter()
```

Memory trick: shouldNotFilter() returning true means “do not run this filter.”

16. Filter vs OncePerRequestFilter

Feature	Filter	OncePerRequestFilter
Origin	Servlet API	Spring Web
Main method	doFilter()	doFilterInternal()
Types	Generic Servlet types	HTTP-specific types
Manual casts	Usually needed for HTTP operations	Not needed in doFilterInternal()
Skip condition	Usually implemented manually	Override shouldNotFilter()
Best suited for	General Servlet filter implementations	Spring web filters needing its dispatch behavior

Choose the abstraction that matches the filter's requirements.

17. Best Practices

- Use filters for cross-cutting concerns rather than business logic.
- Set response headers before downstream processing when possible.
- Return a direct error response when rejecting a request.
- Call chain.doFilter() when processing should continue.
- Call return after writing an error response to prevent downstream execution.
- Do not assume a committed response can still be changed.
- Use wrappers when request or response behavior genuinely needs to be adapted.
- Do not confuse caching a body with automatically replacing it.
- Avoid modifying response bodies in filters unless the requirement clearly justifies it.
- Prefer OncePerRequestFilter when its Spring-specific behavior suits the filter.
- Use shouldNotFilter() to express request-exclusion rules clearly.

18. Interview Questions and Answers

Q1. Can a filter add a response header?

Yes. HttpServletResponse.setHeader() can set a header before the response is committed.

Q2. Why is setting a response header after chain.doFilter() unreliable?

Downstream processing may already have committed the response.

Q3. Can a filter return a response without calling the controller?

Yes. Set the status, write the response body, and return without calling chain.doFilter().

Q4. What does HTTP 401 represent in the example?

It represents an unauthenticated request in the demonstrated token check.

Q5. Why can't a filter simply edit the response body after chain.doFilter()?

The response may already have been written and committed; the filter does not automatically receive an editable business response object.

Q6. What is ContentCachingResponseWrapper?

A Spring Web wrapper that caches response content so it can be inspected before being copied to the underlying response.

Q7. Why is copyBodyToResponse() important?

It copies the cached body to the actual response so it can reach the client.

Q8. Does ContentCachingRequestWrapper automatically replace the request body?

No. It primarily caches content as it is read. Replacing the body generally requires a suitable custom wrapper.

Q9. What is OncePerRequestFilter?

A Spring Web base class that provides once-per-request filter handling within its defined dispatch behavior.

Q10. What does shouldNotFilter() do?

It determines whether a particular request should skip the filter; true means skip.

Q11. What is a request wrapper?

A wrapper delegates to the original request while allowing selected behavior to be overridden for downstream code.

Q12. Should filters routinely modify business response bodies?

Generally no. The controller or service layer should normally determine the business response.

19. Final Revision Flow

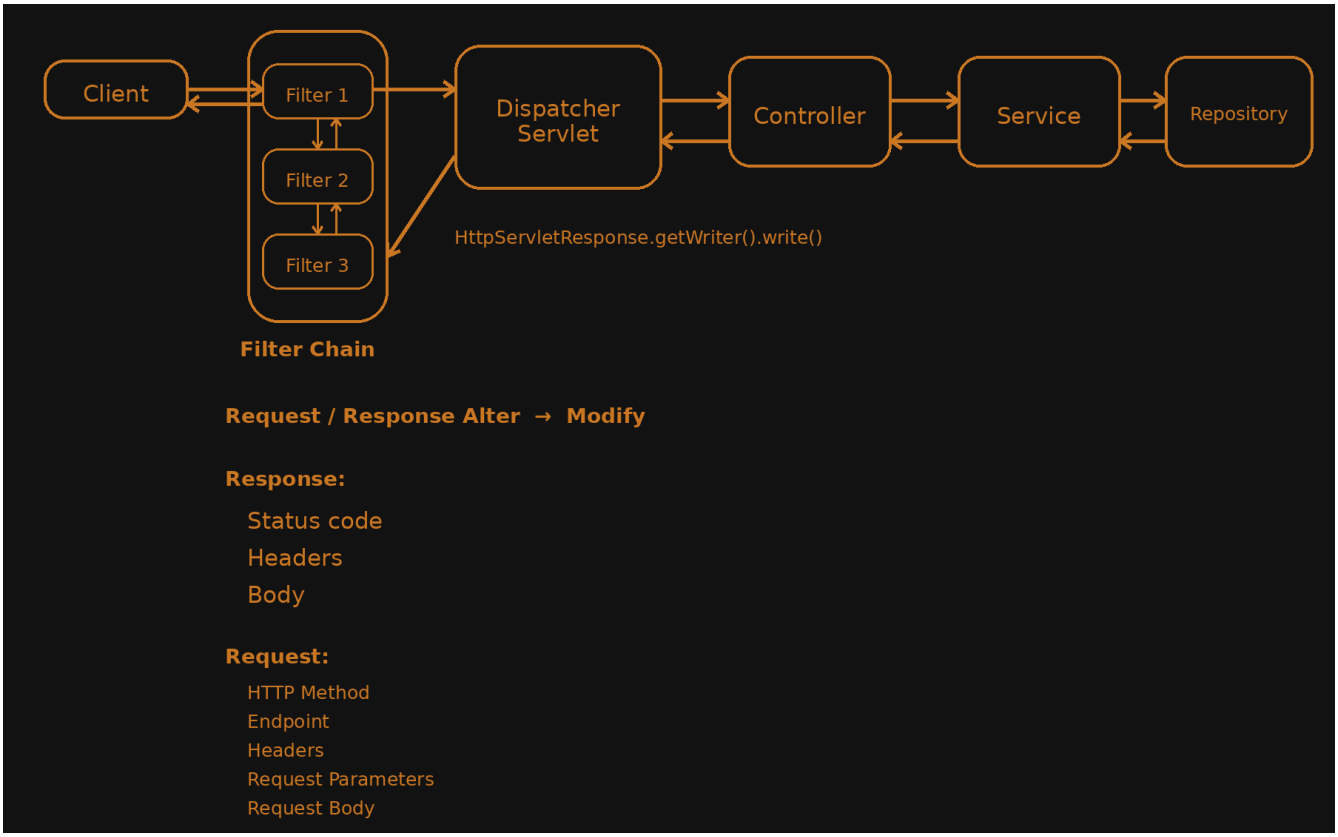
```
Spring Boot Filters
   |
   v
Response Headers
   |
   v
Set headers before response commitment
   |
   v
Direct Error Responses
   |
   v
Set status and content type
   |
   v
Write body and return
   |
   v
Response Body Wrappers
   |
   v
Cache response
   |
   v
Read and modify content
   |
   v
Copy modified content to original response
   |
   v
Request Modification
   |
   v
Original request is not generally mutable
   |
   v
Use a suitable request wrapper when needed
   |
   v
OncePerRequestFilter
   |
   v
doFilterInternal()
   |
   v
shouldNotFilter()
   |
   v
Execute or skip the filter
```

Final takeaway: Filters are powerful for request/response processing, but understanding response commitment, wrapper behavior, and the distinction between caching and replacing data is essential. Use filters for cross-cutting concerns, and keep business response construction in the controller or service layer.

Additional Diagrams — Spring Boot Filters

Diagram 1. Filter chain and request/response flow

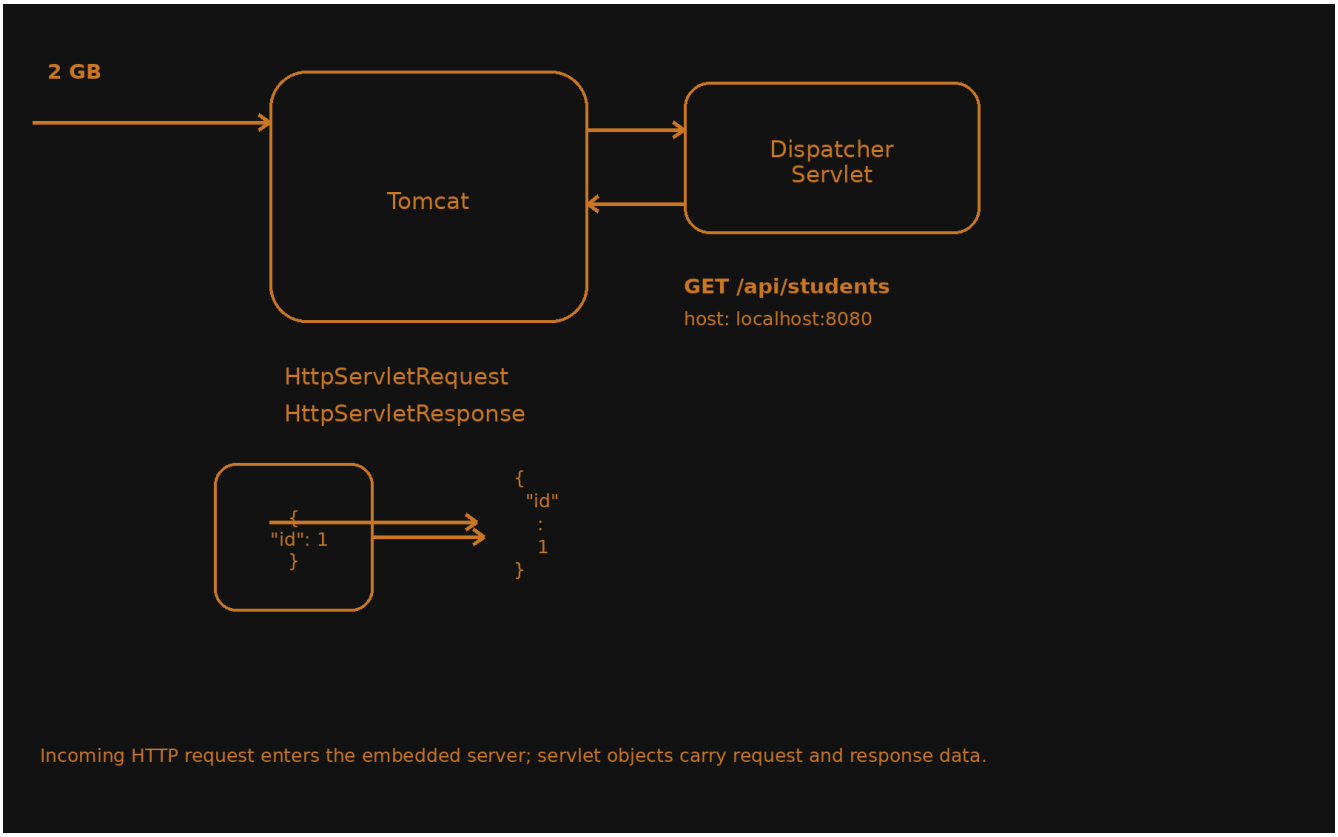
The client request passes through the filter chain before reaching DispatcherServlet. DispatcherServlet routes it to the controller, which calls the service and repository as needed. The response returns through the MVC path and filter chain to the client.



Use this diagram to remember the main request metadata (HTTP method, endpoint, headers, parameters, and body) and response parts (status code, headers, and body). Filters are suitable for cross-cutting request/response handling; business response construction should normally remain in the application layer.

Diagram 2. Embedded server and HTTP request/response

Tomcat receives the HTTP request and provides the servlet request/response objects used by the servlet pipeline. The request includes information such as method, endpoint, headers, parameters, and—when applicable—a body. The response carries a status code, headers, and body.



For response-body modification, a filter cannot reliably edit content after it has already been committed. A response wrapper can cache content for inspection, but body rewriting should be used sparingly because it can change the API contract and add memory overhead.